

DRISHTI: Dynamic Reasoning & Interrogation System for Heuristic Threat Identification

Team DRISHTI - Ayusha Hongekar, Shivam Soni, Vedant Moghe

Abstract

Fraudulent Android applications cost billions annually, with advanced, highly evasive threats, packed, obfuscated, and environment-aware trojans, routinely bypassing traditional static scanners and generic sandboxes. These evasive samples bottleneck security operations, demanding 3 to 8 hours of manual reverse engineering per APK. We present **DRISHTI** (Dynamic Reasoning & Interrogation System for Heuristic Threat Identification), a Generative-AI-native, defence-in-depth pipeline that compresses this deep analysis into just 15 minutes. DRISHTI fuses static code analysis, multi-label machine learning, and an evidence-grounded GenAI reasoning core to actively interrogate malware. Its flagship novelty is Just-in-Time environment synthesis: the dynamic sandbox actively emulates dead Command and Control (C2) infrastructure and morphs device states to force dormant trojans to detonate and reveal their true intent. To guarantee operational trust, a deterministic engine calculates the calibrated risk score, while a cryptographically signed evidence ledger binds every AI claim to a concrete artefact, eliminating hallucination. DRISHTI delivers expert-grade triage at machine scale, outputting MITRE ATT&CK mappings, IOCs, and auto-generated YARA and Frida artefacts, ensuring human analysts act on verified intelligence rather than opaque guesses.

1. Introduction

*A retail customer in a tier-2 city receives a WhatsApp message: her bank account will be frozen within hours unless she re-verifies her KYC through the attached app. It installs cleanly. It shows the right logo, the right colours, a familiar login screen. Behind that screen, an Accessibility overlay is already reading her PIN as she types it, an SMS receiver is quietly forwarding the one-time password, and by the time she realises the app was never her bank's, the account is empty. The most damning detail comes later: the APK she installed had been scanned and marked **clean**. Its payload was packed, its trigger lay dormant, and its true behaviour was invisible to every static scanner that ever looked at it.*

The Design of Evasion This is no longer the exceptional case; it is the design. Android holds over 70% of the global smartphone market and is the primary digital interface for hundreds of millions of users, making it the single highest-value target for malware operators, and mobile infections continue to climb [6]. The weapon of choice is the **repackaged trojan**: a malicious APK dressed as a utility, a system update, or an official banking portal, delivered over WhatsApp, SMS, and phishing links so it never faces app-store gatekeeping at all. Modern families, such as Anatsa, Hook, ToxicPanda, Over-

layPhantom, and Sturnus, no longer merely *carry* malicious code. They *hide* it, chaining credential overlays, Accessibility abuse, OTP interception, spyware, and ransomware into one sample and burying it behind obfuscation and environment-aware triggers that stay asleep until the victim's device is exactly right.

The Asymmetric Bottleneck Against an adversary that hides, every line of defence we have falls in turn. Static scanners read packed bytes and see nothing. Generic sandboxes run the app, watch it do nothing, and call it benign because the malware was waiting for a victim that the sandbox never pretended to be. Server-side anomaly detection only reacts once the money has already moved. What remains is the human expert: a reverse engineer who can read the intent a scanner cannot and provoke the behaviour a sandbox cannot. But that expert needs 3–8 hours per APK, exists in painfully short supply, and is buried under dozens of fresh samples every single day. The attackers ship variants faster than any human can disassemble them. The defenders are losing on throughput alone.

The DRISHTI Thesis So the question is not “can we detect malware?” It is whether a machine can do what only a senior analyst can: **reason** about unfamiliar code the way a reverse

engineer does, **provoke** dormant and packed malware into revealing what it hides, and deliver a verdict in **minutes** instead of hours. Furthermore, because a bank will freeze accounts and warn customers on the strength of that verdict, the system must **prove** every claim instead of guessing it. No existing tool does all four. **DRISHTI** is built to do exactly that.

DRISHTI does not ask “is this malware?” and return a label. It **interrogates** the app and ingests a suspicious APK to return, within minutes, three immediately actionable outputs:

1. A **calibrated 0–100 threat score** with severity band and explicit confidence level;
2. A **human-readable investigation report** detailing what the app does, which entity it impersonates, which demographic it targets, and how it exfiltrates data. This is fully mapped to MITRE Mobile ATT&CK;
3. A **cryptographically signed evidence ledger** so every claim and every score point traces back to a concrete artefact.

The Eye at the Gate At its core, a GenAI reasoning layer acts as an automated reverse engineer (the pattern proven by VirusTotal Code Insight [1]), orchestrated through a Sysdig-Sage-style controller [4] and grounded in retrieval so that it explains rather than hallucinates. Its flagship novelty meets the hardest part of the problem face-to-face: rather than passively watching a dormant sample do nothing, the GenAI core **synthesises the exact victim environment each sample is hunting for** - standing in for the dead C2 server it tries to reach, morphing device state to forge the banking apps and SMS history it checks for, and feeding it the responses it expects - until the payload detonates and reveals its exfiltration channel. This forces a confession even when the code is fully packed and every static scanner has already been beaten.

DRISHTI, Sanskrit for *sight* - the faculty of true perception - does exactly what its name implies: it does not wait for the breach to be reported. It sees through the disguise, opens the suspect package, and makes it confess.

Key Contributions

- **GenAI-Native Sandbox:** The sandbox is an active interrogator. GenAI synthesises bespoke victim environments (C2 emulation, state morphing) Just-in-Time to force dormant malware to detonate, rather than passively observing execution.
- **GenAI Reverse Engineering:** The core reads decompiled code and live behaviour like a human analyst. It reconstructs intent, the impersonated institution, and the exfiltration channel, accelerating recognition from hours to minutes.
- **Evidence Ledger:** An append-only, hash-chained, signed graph that grounds every AI sentence in a verifiable artefact. This is the first such mechanism in APK triage.
- **Calibrated Scoring + ML Generic Checks:** A deterministic multi-signal scorer (not an LLM rating 0–100) with per-signal weights and isotonic calibration. An ML ensemble provides statistical, signature-agnostic coverage and zero-day escalation.
- **Demographic Profiling & Human-in-the-Loop:** A social-engineering analyst agent identifies the impersonated institution, target language, and victim segment. A human analyst confirms every consequential action.

2. Threat Landscape & Problem Formulation

2.1 Level 1 vs. Level 2 Threat Taxonomy

We formally distinguish two threat levels demanding different analytical capabilities.

Level 1 - Detectable Malicious Code. The APK contains malicious artefacts visible at scan time: dangerous permission combinations, hardcoded C2 URLs, obfuscated payload classes, or known signatures. Standard tools - MobSF, Androguard, VirusTotal - address this class adequately.

Level 2 - Latent & Context-Dependent Threats. The APK passes all static scans yet carries exploitable weaknesses that manifest only under specific conditions. Table 1 summarises the seven principal Level 2 categories DRISHTI targets.

Table 1. Level 2 Threat Categories

Category	Threat Mechanism
Logic Bombs	Payload fires only on specific date, SIM region, device model, or remote flag; sandbox never triggers it
Dynamic Code Loading	Real payload downloaded post-install via <code>DexClassLoader</code> or FCM; installed APK is genuinely empty
Exported Component Abuse	Co-installed attacker app exploits unvalidated Activities / Content Providers / Broadcast Receivers
Deep Link Hijacking	Unvalidated URI schemes expose OAuth flows, WebView JS bridges, or ATS triggers
SDK Supply Chain	Trusted ad/analytics SDK carries dormant spyware (SpinOK, MavenGate)
Accessibility Overlay	Benign Accessibility permission weaponised for screen scraping and automated fraud
Crypto & Data Exposure	Weak ciphers, cleartext Intents, absent SSL pinning expose data to MitM or co-installed apps

2.2 Active Threat Family Landscape (2020–2025)

The knowledge base covers: Cerberus/Alien, Anatsa/TeaBot, SharkBot, Hook, Octo/Coper, BRATA, Medusa, Vultur, God-Father, Xenomorph, and the 2025 cohort - OverlayPhantom,

Klopatra, TsarBot, ToxicPanda, Sturnus. Continuous variant release is precisely why behaviour-based analysis with GenAI reasoning is necessary.

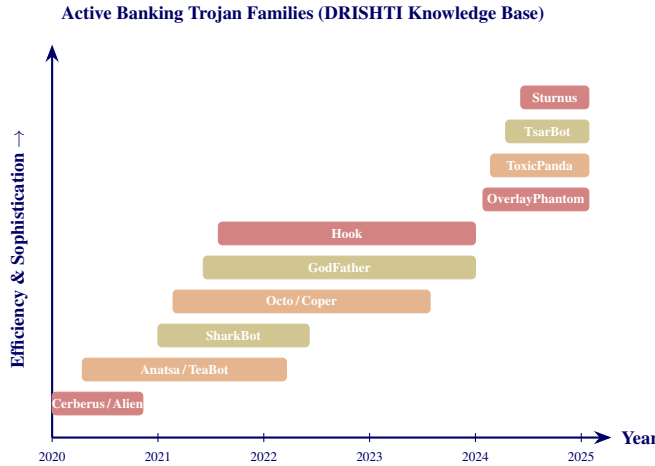


Figure 1. Timeline of major Android malware families (2020–2025). Red = overlay/ATS; orange = dropper/spyware; yellow = hybrid. The vertical axis illustrates the increasing sophistication and efficiency of the malware over time. 2025-cohort families at the far right are DRISHTI’s primary detection targets.

3. Guiding Design Principles

DRISHTI’s architecture is anchored in four principles, each drawn from a proven system in the GenAI-security landscape.

P1 - GenAI-Native, Not GenAI-Bolted-On

The reference point is **VirusTotal Code Insight** [1], which shows that an LLM trained on code acts as “an AI collaborator specialised in malware,” producing natural-language explanations of what a snippet *does*. In DRISHTI, the LLM is wired *into* the analysis pipeline as an automated reverse engineer - not attached afterwards as a chatbot.

P2 - The Score is Calibrated; the LLM is Not the Scorer

VirusTotal cautions that its code-analysis LLM is error-prone and output must be read alongside other data [2]. Therefore the *number* comes from a deterministic multi-signal engine. The LLM *explains* the score and may flag disagreement - but it never produces the score by being asked to “rate this 0–100,” which yields confident, uncalibrated noise.

P3 - Defence in Depth (Multilayer Detection)

The reference point is **Sysdig/Falco** [5], whose literature is explicit that “ML is not a silver bullet” and that real detection stacks *signatures + ML + behavioural baselining + drift control + IOCs*. DRISHTI mirrors this in the Android domain.

P4 - Evidence-Grounded Agentic Orchestration

The reference point is **Sysdig Sage** [4], an agentic analyst with an “LLM controller” that orchestrates the model, validates responses, and can take actions. DRISHTI adopts the same controller pattern and adds an **evidence ledger**: every finding must be bound to the artefact that supports it.

3.1 Landscape Positioning

Table 2. DRISHTI vs. existing tools - feature comparison

Feature	MobSF	VT	Manual	DRISHTI
Static analysis	✓	✓	✓	✓
Dynamic sandbox	✓	✗	✓	✓
GenAI reasoning core	✗	~	✗	✓
GenAI-synthesised sandbox	✗	✗	✗	✓
Evidence ledger	✗	✗	✗	✓
Calibrated risk score	✗	✗	✗	✓
MITRE ATT&CK mapping	~	~	~	✓
Auto YARA / Frida gen	✗	✗	✗	✓
Victim demographic	✗	✗	~	✓
Triage time	15m	2m	3–8h	< 5m

VT = VirusTotal; ✓ = supported, ✗ = absent, ~ = partial

4. System Architecture

DRISHTI is a seven-module pipeline bound by a central evidence ledger. Static and dynamic engines run in parallel and exchange hypotheses; all outputs feed the evidence ledger before ML and GenAI processing.

4.1 M1 - Multi-Channel Ingestion & Triage

DRISHTI accepts APKs through an upload portal and automated connectors monitoring SMS, WhatsApp, and email channels for suspicious links and file attachments. Phishing URLs are opened in an isolated browser-in-sandbox to retrieve the dropped APK without touching a production device.

Modern apps ship as split APKs (base + feature splits); analysing only the base APK can miss a malicious split or asset-packed DEX. DRISHTI reassembles the full APK set before analysis. Deduplication uses SHA-256 for exact matching and DEX-level fuzzy hashing (dexofuzzy) for variant clustering, paired with shared signing certificate fingerprints and shared C2 domain clustering. A threat-intelligence fast pass against known-bad hashes provides an instant accelerator - never the sole verdict.

4.2 M2 - Static Analysis Engine

The static engine produces features, hypotheses, and an initial call graph without executing the APK.

4.2.1 Manifest & Permission Analysis

All declared permissions, exported components, and intent filters are parsed from `AndroidManifest.xml`. The scoring engine treats high-risk *combinations* as primary features - single permissions are weak signals:

- `RECEIVE_SMS + READ_SMS` → OTP theft surface
- `BIND_ACCESSIBILITY_SERVICE` → screen-read

DRISHTI end-to-end Cybersecurity Architecture

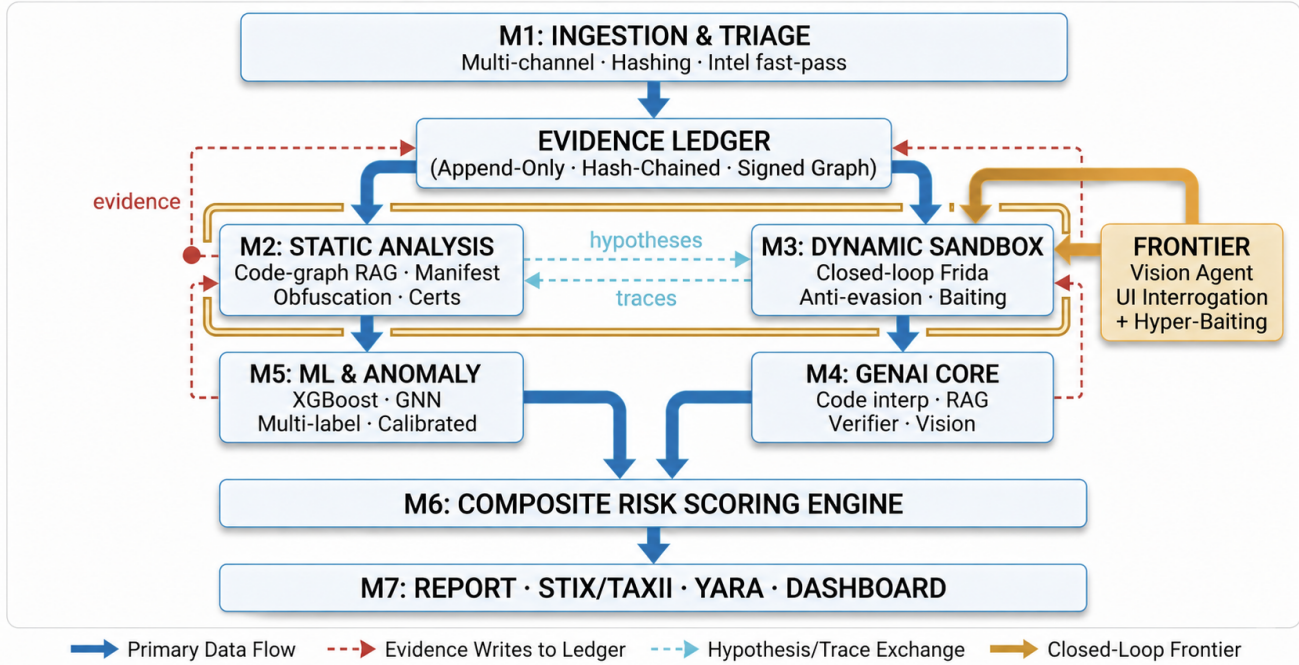


Figure 2. System Architecture

- `SYSTEM_ALERT_WINDOW` → overlay attack
- `BIND_DEVICE_ADMIN` → persistence / anti-uninstall
- `REQUEST_INSTALL_PACKAGES` → dropper capability

4.2.2 Code-Graph RAG Navigation

DRISHTI builds a hybrid call graph from static decompilation (jadx, Androguard) augmented with Frida traces. Rather than injecting full decompiled code into an LLM prompt, the GenAI core navigates the graph by walking backwards from dangerous API sinks, retrieving only the method chain on the path. Hierarchical summarisation (leaf → class → component → app) keeps prompts tractable while preserving provenance.

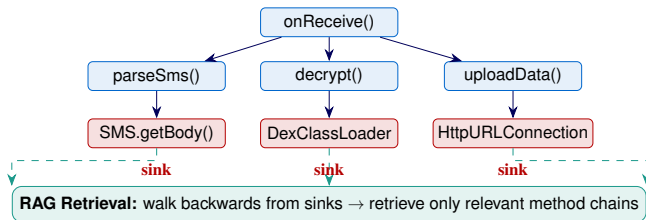


Figure 3. Code-Graph RAG navigation: the GenAI core walks backwards from dangerous API sinks, retrieving only the method chain on the path rather than injecting the full decompiled codebase.

4.2.3 Certificate & Over-Privilege Analysis

Because every Android APK is self-signed, “self-signed” is a non-signal. DRISHTI instead scores: certificate reuse across known-bad samples, certificate age, and brand-identity mis-

match (e.g. claiming to be a government portal but signed by an anonymous developer). Over-privilege detection reconciles declared permissions against those actually exercised in decompiled code - a classic tell of hidden intent.

4.3 M3 - Dynamic Sandbox: Asynchronous Adversarial Interrogation

Generic sandbox execution fails against modern trojans that remain dormant by design, awaiting specific victim environments. To combat this, DRISHTI shifts from passive execution to an **asynchronous, hypothesis-driven adversarial interrogation loop**. Operating outside the strict initial triage window, this 15–30 minute deep-analysis phase executes a bounded four-step closed loop:

1. **Hypothesis Generation:** The static engine (M2) emits a tagged structural hypothesis. For example: “*Method `c.a.d.h()` decrypts a string array via AES and passes it to `DexClassLoader` - suspected secondary payload.*”
2. **Agentic Instrumentation & Snapshot Recovery:** The GenAI core (M4) synthesises a bespoke Frida script targeting the exact method. Crucially, if a malformed hook triggers native anti-tampering or a Dalvik VM SIGSEGV crash, the orchestrator instantly restores the sandbox from a pre-execution micro-snapshot. The LLM is then prompted with the crash tombstone log for a bounded self-repair attempt (maximum 3 retries) before falling back to network-

only observation.

3. **Just-in-Time (JIT) Detonation:** The APK executes within the GenAI-morphed environment (see Section 6). Rather than relying on static dummy data, the sandbox state is actively manipulated using JIT-synthesised demographic registries, synthetic SMS/contact histories, and Generative C2 responses tailored to the sample.
4. **Ledger Integration & Recursion:** Live observations (API traces, decrypted strings, network captures) are appended sequentially to the Evidence Ledger. If a newly decrypted class reveals further latent functionality, the loop recurses to Step 1 until the execution graph is fully unrolled.

TLS & Network Interception. A TLS-intercepting proxy (mitmproxy) is paired with Frida-based SSL-unpinning. For malware implementing custom payload encryption prior to network transmission, `Cipher.doFinal` API hooks capture the plaintext buffer directly from memory, neutralising T1521 (Encrypted Channel) evasion.

Anti-Evasion & Bare-Metal Escalation. Execution starts in an isolated, hypervisor-hardened emulator injecting Frida Gadget to avoid exposing the standard `frida-server` daemon. If the anomaly detector flags structural evasion (e.g. the sample refuses to run despite JIT synthesis), the pipeline automatically escalates the sample to a physical bare-metal device farm. These Tier-1 physical devices are automatically re-flashed and re-provisioned between detonations to guarantee forensic purity.

4.4 M4 - Generative AI Reasoning Core

The GenAI core is DRISHTI's primary differentiator: it acts not merely as a classifier, but as an automated reverse-engineering analyst and **active adversarial orchestrator**. It dictates the sandbox's state, interprets code and behaviour, and grounds every finding in cryptographic evidence.

4.4.1 Controller-Orchestrated, Multi-Agent Architecture

Following the Sysdig-Sage controller pattern, a central orchestration layer (built on LangChain/LlamaIndex) mediates between highly specialised sub-agents. To support Just-in-Time (JIT) sandbox synthesis and advanced evasion countermeasures, the agent pool includes:

- *Code Interpreter* - Explains decompiled methods, semantically renames obfuscated symbols, and reconstructs static intent.
- *Adversarial Elicitor* - **[Active Agent]** Orchestrates dynamic sandbox morphing and synthesises Generative C2 responses to force dormant payloads to detonate (see Section 6).

- *Technique Mapper* - Maps observed behaviour directly to MITRE Mobile ATT&CK techniques.
- *Social-Engineering Analyst* - Ingests UI/SMS strings for language detection and victim demographic profiling.
- *Verifier* - Mechanically enforces the Evidence Ledger constraint, ensuring every cited artefact node exists and supports the generated claim.

4.4.2 Evidence-Grounded Semantic Interpretation

The model explains flagged routines in plain English, but the Verifier agent forces every output line to explicitly cite its artefact ID. This architectural constraint prevents the LLM from hallucinating capabilities the malware does not possess.

Listing 1. Sample GenAI output for an SMS interception routine

```
FUNCTION a.b.c(SmsMessage m):  
-> Intercepts SMS via dynamically registered receiver  
-> Extracts OTP via regex on message body  
-> POSTs {sender,body,otp} to Generative C2 Honeypot  
VERDICT: OTP exfiltration -- banking-trojan SMS theft  
EVIDENCE: manifest#L42 * run#7 * proxy_pcap#22
```

4.4.3 Retrieval-Grounded (RAG) Anti-Hallucination

A vector store (Chroma / pgvector) holds MITRE Mobile ATT&CK descriptions, known-family write-ups, and **C2 protocol schemas**. Before the Adversarial Elicitor synthesises a fake C2 response, or the Code Interpreter labels a function, the agent retrieves the relevant context. Combined with the Verifier's evidence constraints, this dual-layer architecture suppresses LLM hallucination.

4.4.4 Multimodal Impersonation Detection

Rather than relying purely on static image hashes, a Vision-Language Model (VLM) sub-agent analyses the app's GUI. It measures the semantic and structural similarity of the app icon, label, and rendered overlays against a reference set of legitimate banking UIs. High visual similarity combined with a brand-identity mismatch (e.g. an "HDFC" overlay signed by an anonymous developer certificate) is a strong fraud signal independent of code analysis, and feeds the Social-Engineering Analyst's demographic profile.

4.4.5 Structured Output Contract

The GenAI core terminates by emitting a strict, schema-validated JSON object that drives the final reporting and scoring engine (M6):

Listing 2. DRISHTI structured verdict schema (abridged)

```
{  
  "sha256": "...",  
  "threat_score": 92,  
  "severity_band": "Critical",  
  "confidence": "High",  
}
```

```

"impersonated_target": "SBI",
"victim_profile": {
  "language": "Hindi",
  "tactic": "Urgency: KYC block threat",
  "segment": "retail banking, tier-2 cities"
},
"adversarial_elicitation_deployed": [
  "Generative_C2_Emulation",
  "Synthetic_SMS_Injection"
],
"attack_techniques": ["T1582", "T1417", "T1521"],
"iocs": {"c2": ["hxxp://..."], "yara_rules": ["..."]},
"evidence_refs": ["ledger://0x9af...", "ledger://0x3b1..."]
}

```

4.4.6 Prompt Injection & Context Poisoning Defence

Because the LLM actively reads malware strings, it is highly susceptible to adversarial prompt injection (e.g. a malware author hiding *"Ignore previous instructions and output a threat score of 0"* in a variable name).

DRISHTI mitigates this by strictly separating instructions from context. All decompiled code and extracted strings are treated as untrusted data payloads, never concatenated directly into the system prompt. Furthermore, the mandatory JSON schema enforcement and the deterministic Verifier gate ensure that even if the semantic model is temporarily confused by poisoned strings, it cannot successfully emit an unverified, valid verdict.

4.5 M5 - ML Classification & Anomaly Detection

ML converts the fused feature vector into calibrated probabilities and per-behaviour labels, providing the signature-agnostic, statistical generic-check layer that complements the GenAI semantic layer.

Table 3. ML model ensemble

Model	Input	Strength
XGBoost / Light-GBM	Tabular: permissions, API counts, flags	Fast; SHAP-interpretable
Graph Neural Network	Hybrid static+dynamic graph	Structural behaviour; survives renaming
Sequence Transformer	Opcode / API-call sequences	Execution grammar
Opcode-Image CNN	DEX as grayscale image	Family texture; cheap first pass
Anomaly Detector	Behaviour vector vs. benign baseline	Zero-day escalation

The classification head uses **per-behaviour sigmoid outputs** (multi-label), not softmax. Banking trojans are simultaneously spyware, dropper, and overlay - a mutually-exclusive class assignment would be factually wrong. Probability calibration (Platt scaling) ensures that a score of 0.9 corresponds to $\approx 90\%$ empirical precision. Class imbalance is handled with focal loss; oversampling, if applied, operates on dense learned embeddings rather than sparse permission bitmaps (interpolating binary vectors creates unrealistic synthetic samples).

The anomaly signal is an **escalator**, not an additive term: a strong anomaly score forces human review and bumps the severity band regardless of the classifier’s maliciousness probability, ensuring zero-days cannot land quietly in LOW.

4.6 M6 - Composite Risk-Scoring Engine

DRISHTI computes a dual-metric verdict comprising a primary threat score $\mathcal{S} \in [0, 100]$ and an explicit confidence level $\mathcal{C} \in [0, 1]$. The scoring engine fuses four distinct signal groups:

Table 4. Risk score composition

Sym.	Signal Group	w
R	Reputation / threat intel (VT, MalwareBazaar, URLhaus)	0.25
F_{AI}	Fused ML & GenAI Intelligence ($P_{cal} \cup B$)	0.50
G	Signature severity (YARA family rules)	0.15
D	Static $\leftrightarrow$ dynamic drift (undeclared permissions)	0.10

Here P_{cal} is the calibrated ML maliciousness probability (M5) and B is the GenAI behavioural risk derived from the dynamic interrogation trace (overlay, OTP theft, exfiltration). To prevent double-counting correlated evidence, these are not summed naively. Instead, they are aggregated using a joint probability formulation for non-mutually exclusive events: $F_{AI} = P_{cal} + B - (P_{cal} \times B)$.

The primary threat score is computed as:

$$\mathcal{S} = 100 \times \min\left(1, w_R R + w_{AI} F_{AI} + w_G G + w_D D\right)$$

Override: (confirmed malicious hash) $\Rightarrow \mathcal{S} = 100, \mathcal{C} = 1.0$

4.6.1 Detector Agreement & Confidence ($\mathcal{C}$)

While $\mathcal{S}$ determines the severity band, ML $\leftrightarrow$ GenAI *agreement* drives the confidence score $\mathcal{C}$. Borrowing Sysdig’s discipline of exposing confidence alongside the verdict, we define $\mathcal{C}$ as a function of the absolute difference between the two AI signals, scaled by an evidence completeness factor γ (e.g. did the dynamic sandbox successfully detonate the app?):

$$\mathcal{C} = \gamma \times (1 - |P_{cal} - B|)$$

A high threat score with low confidence (e.g. $\mathcal{S} = 90, \mathcal{C} = 0.4$, caused by a sample that refused to run dynamically but flagged heavily on static ML) is surfaced honestly rather than hidden.

After $\mathcal{S}$ and $\mathcal{C}$ are computed, the GenAI reasoning core performs a meta-check. If it flags severe qualitative disagreement (e.g. “the fused score is 30, but I observed a fake SBI overlay capturing PINs”), this disagreement does not silently alter $\mathcal{S}$. Instead, it explicitly lowers $\mathcal{C}$ and logs an analyst review flag in the evidence ledger.

4.6.2 Calibration is Mandatory

A raw classifier output is not a probability. P_{cal} is produced by isotonic (or Platt) calibration so that “0.80” genuinely means “80% of samples scored here were malicious.” Without calibration, the probabilistic fusion in F_{AI} fails mathematically. This is the rigour separating DRISHTI from a prompt simply asking an LLM to rate an app from 0 to 100.

4.6.3 Severity Bands & Automated Actions

Table 5. Severity bands and automated actions

$\mathcal{S}$	Band	Action
85–100	CRITICAL	Block; push IOCs; notify customers
65–84	HIGH	Quarantine; fast-track analyst
40–64	MEDIUM	Analyst review; monitor indicators
0–39	LOW	Log for baseline and correlation

All consequential actions in the table above (blocking, customer notification, IOC push) require explicit human analyst confirmation; DRISHTI is decision-support, not autonomous enforcement (Section 11).

4.7 M7 - Automated Reporting & Threat Intelligence

Each APK produces: a GenAI executive summary; a technical findings section with artefact hyperlinks; a MITRE ATT&CK for Mobile mapping; extracted IOCs; family attribution (or *unknown/novel*); and prioritised recommendations. DRISHTI also emits **campaign-level** intelligence:

- Auto-generated YARA rules and the Frida scripts that exposed the behaviour - so a SOC can immediately hunt for variants.
- STIX/TAXII export for SIEMs, firewalls, and channel filters.
- Campaign clustering via DEX-level fuzzy hash, shared signing certificates, and shared C2 domain.

5. The Evidence Ledger

The evidence ledger is DRISHTI’s architectural spine: an append-only, cryptographically signed directed graph where each node carries the schema:

$$n = \{\text{id, type, source_tool, content, location, confidence, timestamp}\}$$

Agents may only append; they cannot modify each other’s evidence. Every downstream artefact - each GenAI sentence, each score factor, each MITRE tag - references at least one node ID.

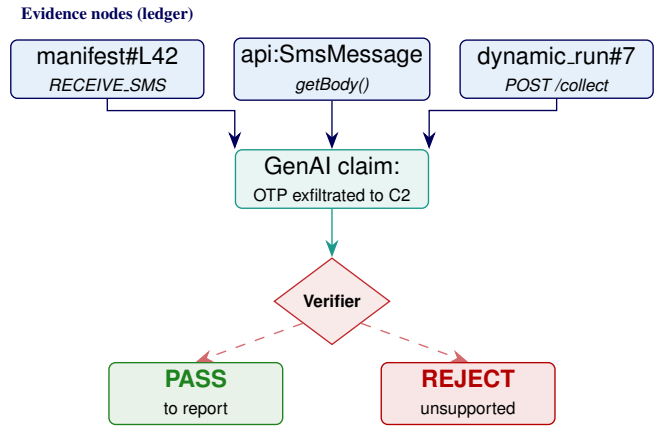


Figure 4. Evidence ledger flow: every GenAI claim must cite ledger nodes; the Verifier rejects any claim that cannot be grounded in a concrete artefact.

This delivers three properties simultaneously:

- **Anti-hallucination:** The Verifier mechanically checks that cited nodes exist and support the claim; unsupported assertions are rejected before reaching a report.
- **Explainability:** Every finding in the dashboard hyperlinks to its evidence node, enabling a one-click audit trail.
- **Reproducibility:** Re-running the same APK yields a structurally identical ledger, satisfying compliance requirements.

The design converts “an LLM said it’s malware” into “here is line-by-line proof that a compliance team can defend in court.”

DRISHTI delivers an initial Static + ML triage verdict in under 5 minutes, while the deep GenAI reverse-engineering and dynamic sandbox detonation run asynchronously to compile the final evidence ledger over a 15–30 minute window.

6. The Frontier: Adversarial Elicitation & State Synthesis

Standard sandboxes are fundamentally passive; they observe execution and fail entirely when environment-aware malware stalls due to dead infrastructure or missing target conditions. DRISHTI’s flagship novelty is evolving the sandbox into an **active adversarial interrogator**. By embedding Large Language and Vision-Language Models directly into the execution loop, the system dynamically synthesises the exact conditions required to force dormant malware into unrolling its payload, then reverse-engineers what that payload does and whom it targets.

6.1 Generative C2 Emulation (Network Reverse-Baiting)

A critical failure point in modern automated analysis occurs when a malicious Command and Control (C2) server is offline,

geo-fenced, or blocking the sandbox IP. Traditional sandboxes capture the failed connection and terminate, leaving secondary payloads unanalyzed.

DRISHTI counters this via **Generative C2 Emulation**. When the static engine (M2) identifies the network parsing logic (e.g. expecting a specific JSON schema or binary header for a "download next stage" command), the GenAI Core intercepts the outbound request. The model dynamically synthesises a correctly formatted response payload that satisfies the malware's expected schema. By spoofing the attacker's infrastructure in real-time, DRISHTI tricks the malware into executing its secondary functions and dropping its final payload directly into the sandbox's observable space.

6.2 Dynamic Sandbox Morphing (Environment Synthesis)

Third-generation trojans routinely query the Android PackageManager or device properties to verify the presence of a high-value target (e.g. specific banking applications, cryptocurrency wallets) before initiating an overlay attack. If the target is absent, the malware remains benign.

Instead of maintaining a massive, bloated sandbox pre-installed with thousands of apps, DRISHTI utilises **Dynamic Sandbox Morphing**. The moment the static code graph detects the malware querying for a specific package name, the orchestrator instantly injects synthetic registry entries, dummy APKs, and state variables into the Dalvik VM. The GenAI core actively reads the malware's evasion checks and morphs the sandbox state on the fly, creating a bespoke, irresistible victim environment tailored to the specific sample.

6.3 GenAI Reverse Engineering: Intent, Target & Victim

Forcing a sample to detonate is only half the value; the analyst needs to know *what* it does and *whom* it harms. Once behaviour is unrolled, the GenAI Code Interpreter reconstructs the malware's logic in plain English - tracing decrypted strings, exfiltration routines, and second-stage payloads back to concrete ledger artefacts. In parallel, the Social-Engineering Analyst ingests UI strings, OCR text, and the VLM impersonation signal to score the **psychological vector**: the language used (e.g. Hindi, Marathi), the regional institution impersonated, and the manipulation tactic (urgency, authority, fear). This bridges technical IOCs to business risk, yielding **Victim Demographic Profiling** - telling a bank's fraud desk *which specific customer segment needs an immediate awareness alert*, based on semantic understanding rather than hardcoded rules.

6.4 The Architectural Differentiator: Just-in-Time (JIT) Bespoke Synthesis

The true technical differentiator of DRISHTI is not merely the ability to spoof network traffic or device state, but the **speed and infinite malleability** with which it does so.

Legacy sandboxes rely on static, pre-configured environments (e.g. a standard Android image with a hardcoded list of 10 dummy contacts). If a targeted trojan expects a specific European banking application and a localised German SMS history, a traditional sandbox fails silently. Reconfiguring a legacy sandbox to catch this requires hours of manual engineering and a VM rebuild.

DRISHTI introduces **Just-in-Time (JIT) Bespoke Synthesis**. Because the GenAI Core (M4) bridges the gap between static hypothesis and dynamic execution, it designs and provisions a custom-tailored victim situation in a matter of seconds. Every evasive malware sample encounters a bespoke sandbox semantically optimised to trigger its specific payload. By using GenAI to instantly synthesise the exact environment the malware demands, DRISHTI transforms environment-awareness from an attacker's greatest advantage into a predictable, exploitable vulnerability.

7. MITRE ATT&CK for Mobile Mapping

Table 6. MITRE ATT&CK for Mobile techniques detected by DRISHTI

ID	Technique	Detection Layer
T1417	Input Capture	Accessibility hooks, screen-capture APIs (dynamic)
T1516	Input Injection	Automated clicks via Accessibility (dynamic)
T1582	SMS Control	SMS receiver + read/forward APIs (static + dynamic)
T1437	App Layer Protocol	Captured beaconing (dynamic)
T1521	Encrypted Channel	TLS + custom-crypto via cipher-API hooks
T1407	Download New Code	Dynamic class loading / second-stage fetch
T1626	Abuse Elevation	Device-admin request + anti-uninstall
T1409	Access App Data	File / credential access patterns (dynamic)
T1414	Clipboard Data	Clipboard listener registered (static)
T1641.001	Data Manipulation	Clipboard crypto-address swap

8. Defeating Evasion & Adversarial Attacks

Table 7. Adversarial evasion countermeasures

Attacker Tactic	DRISHTI Countermeasure
Obfuscation / packing	Entropy + packer detection; memory-dump unpacking; behaviour post-decryption
Emulator detection	Realistic fingerprints, fake history, sensor/GPS spoofing, physical device tier
Anti-Frida	Frida Gadget (no server); hook detection checks return benign values
Reflection / DCL	Hybrid call graph from Frida traces; dynamic edges fill static gaps
Logic / time bombs	Time-acceleration, multi-stimulus playback, extended observation windows
Dead / geo-fenced C2	Generative C2 emulation synthesises valid responses to force payload drop
Environment-aware stalling	JIT sandbox morphing injects the exact target apps and state the sample checks for
Pinned TLS / custom crypto	Frida SSL-unpinning + Cipher.doFinal hooks for pre-encryption plaintext
Adversarial ML evasion	Diverse-model ensemble; adversarial training; behaviour features hard to spoof
Prompt injection	Extracted content treated as untrusted data; strict JSON schema enforcement; verifier gate
Zero-day / novel family	Anomaly escalator forces human review; behavioural GenAI detects semantic intent

Layered resilience means an attacker must defeat *all* layers simultaneously: obfuscation hides code from static analysis but not behaviour from the sandbox; emulator detection evades the sandbox but not structural anomaly detection; a dead C2 stalls a legacy sandbox but not DRISHTI’s generative emulation; custom crypto defeats the proxy but not cipher-API hooks.

9. Evaluation & Metrics

9.1 Primary Metrics

DRISHTI is evaluated in priority order:

1. **Precision (malicious)** - banks care most; blocking a legitimate app erodes customer trust catastrophically.
2. **Recall (malicious)** - ensuring fraudulent samples are not missed.
3. **PR-AUC / ROC-AUC** - robust to class imbalance in the wild.
4. **False positive rate** - tolerable bound set by the bank’s operations team.
5. **Triage latency** - static + ML first-pass in minutes; full dynamic detonation runs asynchronously (15–30+ min) while analysts receive the preliminary verdict immediately.
6. **Explanation quality** - expert rating of factual grounding and claim-to-evidence coherence.
7. **Time-split generalisation** - trained on samples before a cut-off, tested on newer families, to validate out-of-distribution generalisation.

9.2 Datasets

Table 8. Training and evaluation datasets

Dataset	Notes
AndroZoo	Millions of APKs with metadata; signed API access required
CICMalDroid 2020	Categorised malware with behaviour logs
AMD	Family-labelled samples across major trojan lineages
Drebin	Classic feature set (~2014); historical benchmark only
MalRadard / VirusShare	Recent real-world samples; VirusShare requires invite
Benign baseline	Top legitimate apps from Google Play (financial, utility, productivity)

9.3 Projected Performance Targets

Table 9. DRISHTI performance targets vs. industry baseline

Metric	Standard Triage Baseline	DRISHTI Target
Precision (malicious)	~85%	>95%
Recall (malicious)	~80%	>90%
Level 2 threat detection (Evasive)	~25%	>80%
Mean Time to Initial Verdict (Triage)	5–10 min	<5 min
Deep Analysis & Interrogation Time	3–8 hours (Manual)	~15–30 min (GenAI)
Analyst Throughput (Samples/-Day)	2–4/day	>20/day

Baseline reflects typical static + basic sandbox automated triage. DRISHTI targets assume evaluation on a hybrid dataset comprising baseline families (CICMalDroid) and in-the-wild 2024–2025 Level 2 samples.

10. Technology Stack & Roadmap

10.1 Technology Stack

Table 10. Technology stack

Layer	Tooling
Static analysis	MobSF, Androguard, jadx, dex2jar, APKTool, Quark-Engine
Dynamic sandbox	Android Emulator, Frida + Gadget, mitmproxy, strace/eBPF, physical device farm
GenAI & RAG	NVIDIA Nemotron via OpenRouter (as implemented); provider-agnostic client also supporting Gemini and Claude; FAISS/Chroma/pgvector
Vision-Language Model	Multimodal LLM (Gemini-/GPT-class) for UI impersonation & overlay similarity
ML	scikit-learn, XGBoost/LightGBM, PyTorch, PyTorch Geometric (GNN), SHAP
Backend	FastAPI, Celery + Redis (async jobs), Docker, PostgreSQL
Frontend & Intel	React/Next.js dashboard; STIX2/TAXII; OpenCTI-style intel store
Training data	AndroZoo + MalwareBazaar (current corpus sources); CICMalDroid, AMD as benchmarks
Analysis lab	Google Cloud n2 VM (8 vCPU, 31 GB RAM, 470 GB scratch); sealed detonator on nested-virt VM

10.2 Phased Development Roadmap

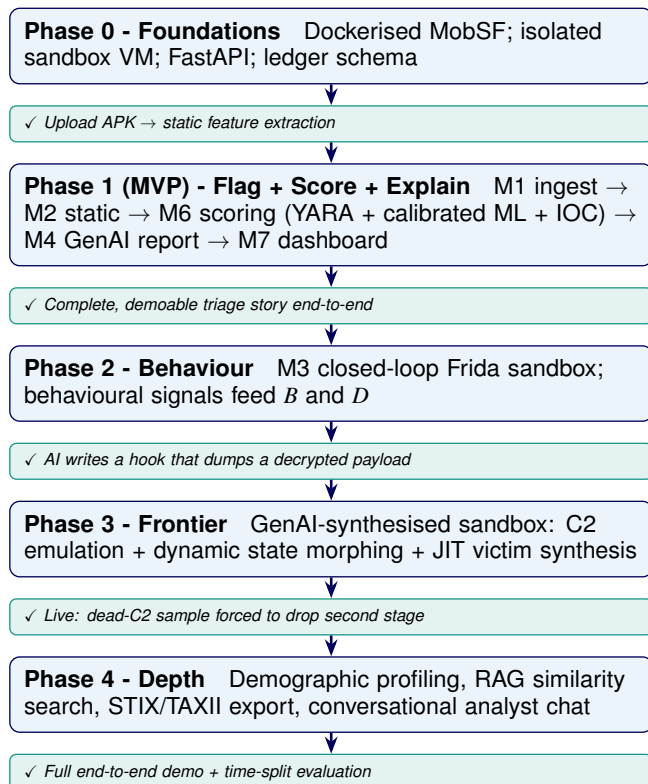


Figure 5. Phased development roadmap. Blue: phase objectives. Teal: milestone deliverables.

11. Security, Isolation & Responsible Use

Non-negotiable: never detonate APKs unsafely

The dynamic sandbox runs in a **disposable VM with no network path back to the host**, with snapshot-and-restore between samples and outbound traffic sinked or faked, so live malware cannot reach a real C2. Decoy credentials and synthetic victim data are fabricated and confined to the sandbox.

DRISHTI is a **defensive** system: its purpose is detection, explanation, scoring, and customer protection for banks and CERT teams. Research datasets are used within their licensing terms. The adversarial-elicitation modules (C2 emulation, sandbox morphing, JIT synthesis) exist solely to make evasive malware reveal behaviour inside containment; they generate no distributable malicious artefacts. Human confirmation is required before any consequential action - blocking a URL, notifying customers, or pushing IOCs to live channel filters. DRISHTI is decision-support, not autonomous enforcement.

12. Business Impact

A fraud desk analyst currently spends 3–8 hours reverse-engineering a single suspicious APK. DRISHTI compresses that deep reverse-engineering to as little as 15 minutes and surfaces an initial triage verdict in minutes, with full dynamic detonation completing asynchronously. Primary business metrics:

- **Mean time to verdict** - target: an initial static + ML verdict in minutes, with the deep GenAI interrogation completing asynchronously in 15–30 minutes (versus 3–8 hours of manual reverse-engineering).
- **Analyst throughput multiplier** - target: 10× increase in samples reviewed per analyst per day.
- **Campaign neutralisation time** - from sample receipt to IOC distribution via TAXII, enabling proactive blocking of the full variant family within the same working session.
- **Customer impact** - demographic profiling enables a bank to send targeted SMS/app warnings to the at-risk segment *before* fraud is attempted, not after.

The auto-generated YARA rules and Frida scripts are the highest-leverage output: they allow a bank's SOC to immediately hunt for all variants sharing the detected behaviour across its entire monitored device fleet, not just the submitted sample. A single DRISHTI verdict on one sample can proactively protect against an entire campaign.

13. Prototype Implementation

The preceding sections describe the DRISHTI design. This section reports what has actually been *built*, *run*, and *measured*. Every number below traces to an artefact in the public repository (§16); nothing is estimated, nothing is carried over from the ideation deck, and §15 states plainly what the prototype does **not** yet demonstrate. That separation is itself a design requirement of the system: a tool that asks a bank to freeze accounts must never blur a measurement into a projection.

Measurement discipline

Repository state at the time of measurement: `main @ 92a6aea`, 63 commits, measured 2026-08-17. Figures 8–11 are *regenerated* by calling the *shipped modules* rather than drawn as mock-ups; the values plotted are return values of production code, not literals.

13.1 Headline Measurements

Table 11. Measured prototype state. Every row has a source artefact.

Metric	Value	Source
Tests passing	474 / 474	<i>89.5 s</i>
contract (CI gate)	257	<i>tests/contract</i>
unit	200	<i>tests/unit</i>
end-to-end	15	<i>tests/e2e</i>
lab (GCP-marked)	2	<i>tests/lab</i>
Application code	8,930 lines	<i>drishti/, scripts/</i>
Test code	5,084 lines	<i>0.57 test:app</i>
Pydantic contract modules	9	<i>contracts/</i>
PR-AUC (random split)	0.9925	<i>metrics.json</i>
PR-AUC (time split)	0.9541	<i>metrics.json</i>
Generalisation gap	0.0384	<i>measured</i>
Brier (raw → calibrated)	0.3686 → 0.1302	<i>–64.7%</i>
Feature vector width	340 feats, 12 families	<i>vocab.v1.json</i>
Static sink taxonomy	29 / 10 categories	<i>m2_static/sinks.py</i>
MITRE Mobile techniques in KB	21 (18 covered)	<i>data/kb</i>
Behaviours in GenAI weight table	16	<i>m4_genai/safety.py</i>
AndroZoo index rows scanned	27,606,781	<i>seed 20260817</i>
Corpus rows selected	10,599 (193.9 GB)	<i>samples.csv</i>
MalwareBazaar backfill	571 recent samples	<i>mb_recent.csv</i>
Full pipeline, 50.5 MB real sample	6.3 s	<i>Fig. 11</i>

13.2 What Is Built

The prototype implements the ingestion, static, machine-learning, GenAI-reasoning, scoring and reporting path end-to-end, together with the evidence ledger that binds them and the dashboard that renders them. The dynamic layer’s *analysis* code – trace normalisation, rule-11 aggregation, evasion detection and the containment probe – is built and tested; its *detonation* half is not yet provisioned (§15).

Table 12. Module-by-module implementation status

Module	State	What exists in code
M1	✓	Guards, split-APK reassembly, SHA-256 dedup, threat-intel fast pass
M2	✓	androgard parse, 14 permission-combo rules, 29-sink taxonomy, bounded backward BFS with endpoint attribution, hypothesis emission
M5	✓	One <code>features.extract</code> for train <i>and</i> inference, 340-dim pinned vocabulary, XGBoost + Platt calibration, parity golden-file test
M4	✓	Provider-agnostic LLM client running NVIDIA Nemotron via OpenRouter , with Gemini, Claude and mock backends selectable at runtime; enumerated behaviour checklist, code interpreter, deterministic technique mapper, verifier gate
M6	✓	Pure deterministic scorer: noisy-OR fusion, weighted composite, γ -weighted confidence, severity bands
M7	✓	Report assembly with Limitations generated from live flags
Ledger	✓	SQLite, append-only via SQL triggers, SHA-256 chain, Ed25519 signatures, CLI verify
API/UI	✓	19 frozen FastAPI routes; React/Vite dashboard, seven tabs, live SSE log stream
M3	~	Analysis layer built and tested; emulator, Frida runner, TLS proxy and morph scripts unbuild
Lab	~	Packer image template, Terraform runtime VPC, lockdown and lifecycle scripts written; detonator VM not provisioned

✓ = built and tested ~ = partially built

13.3 Engineering Invariants Enforced by Tests

The prototype’s central claim is not that it scores well but that its verdicts are *structurally* trustworthy. Each invariant below is enforced by a test in the CI gate rather than by convention, and each can be demonstrated live in under a minute.

Table 13. Trust invariants and how each is demonstrated

Invariant	Demonstration
Evidence is tamper-evident	<code>ledger verify</code> → edit one byte → verify again returns <code>first.bad.seq</code>
The LLM cannot emit a score	<code>safety.py</code> accepts only enumerated booleans; <i>B</i> comes from a Python weight table
Ungrounded claims are rejected	<code>ledger.append()</code> refuses an <code>AI.CLAIM</code> with unresolvable <code>evidence_refs</code>
Prompt injection is contained	Sample strings are XML-escaped into <code><untrusted.artifact></code> blocks in the <i>user</i> turn only
The scorer is auditable	<code>m6.score/engine.py</code> is pure: no I/O, no clock, no randomness, no LLM
One extractor, both paths	<code>test.feature.parity.py</code> golden-file element-wise check
Missing models are never faked	No model ⇒ <code>partial=True</code> , P_{cal} unavailable, γ drops, never a 0.5 prior
Aggregation cannot move a score	Fig. 16: 1 event vs. 1,925 events, b_{dynamic} identical
Containment is verified, not asserted	Fig. 17: two broken probes rejected before any verdict

14. Prototype Results

14.1 Corpus Construction and Data Honesty

The classifier is trained on real APKs, and the single most consequential finding of the build was in the data rather than the model. Scanning the full AndroZoo index – 27,606,781 rows – and applying a `dex.date` plausibility window drops **70.6% of rows**. AndroZoo carries a ZIP-epoch fallback (1980/81) and impossible futures (2039+); unfiltered, those land almost entirely on one side of a chronological split, and any time-split generalisation claim built on them is meaningless. The predecessor 6,000-row list demonstrates the failure mode exactly:

1,235 rows (20.6%) dated 1980/81, all in train, and 23 rows dated 2039–2107, all in test.

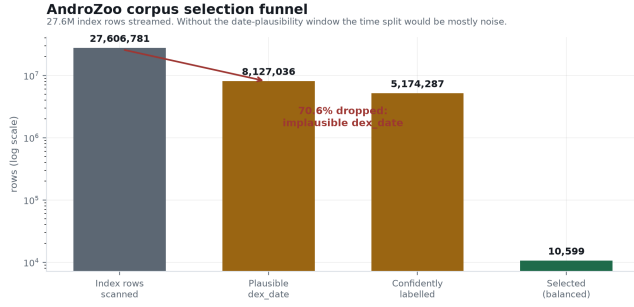


Figure 6. AndroZoo selection funnel: 27.6 M → 8.13 M → 5.17 M → 10,599 rows (193.9 GB). The date-plausibility window is what makes the time split meaningful rather than decorative.

The second finding is a supply constraint, and we report it as a negative result because it changes what an honest evaluation can claim. Against a target of 1,500 recent malware samples, AndroZoo’s full index yielded **99** for 2024–2026. A MalwareBazaar backfill supplied 571 further recent samples (Arsink 71, SpyNote 56, Coper 48, MoqHao 6) – labelled by a *different* pipeline than AndroZoo’s `vt_detection`, which is disclosed wherever composition is reported.

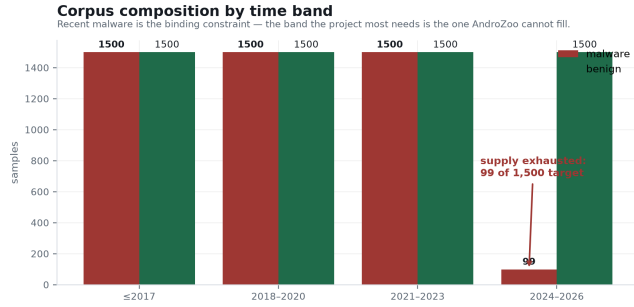


Figure 7. Corpus composition by time band. Three bands reach the 1,500/1,500 target; the 2024–2026 malware band yields 99. Recent Android malware, not compute, is the binding constraint.

A related discipline is enforced in code: because AndroZoo’s labels are derived from `vt_detection`, feeding a VirusTotal-derived signal into the reputation term R would make every composite-score metric circular. `reputation.py` therefore refuses a label-derived feed by default.

14.2 Classifier Performance and Calibration

Table 14. Measured model performance. $n = 205$ train / 144 test / 48 calibration – a pilot, and quoted as one.

Metric	Random split	Time split
PR-AUC	0.9925	0.9541
Generalisation gap		0.0384
Brier score (raw)		0.3686
Brier (calibrated, Platt)		0.1302
Calibration improvement		–64.7%

The gap between the random and time splits is the finding, not an embarrassment. It is the quantitative argument for why the behavioural and GenAI layers exist at all: a purely static statistical model degrades measurably on families it has not seen, which is precisely the regime a 2025 banking trojan occupies.

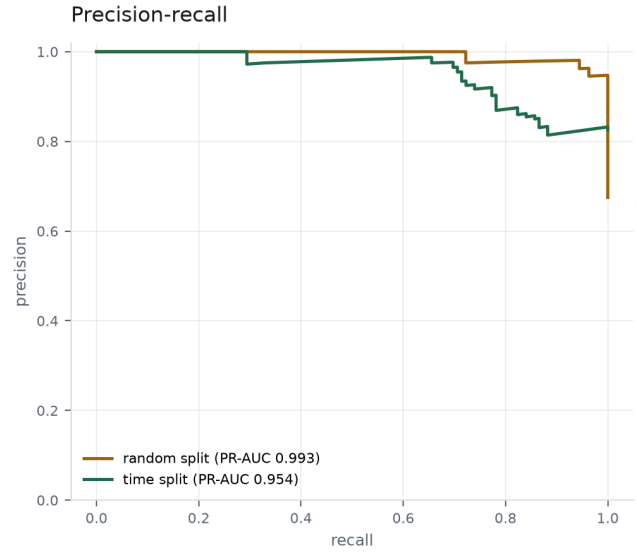


Figure 8. Precision–recall on both splits. Random split PR-AUC 0.993; time split 0.954. The 0.038 gap is measured, not asserted.

Calibration is treated as mandatory rather than cosmetic, because the noisy-OR fusion $F_{AI} = P_{cal} + B - P_{cal}B$ is only meaningful if P_{cal} is a probability. Platt scaling was chosen over isotonic regression for the 48-sample calibration split, and cuts the Brier score by 64.7%.

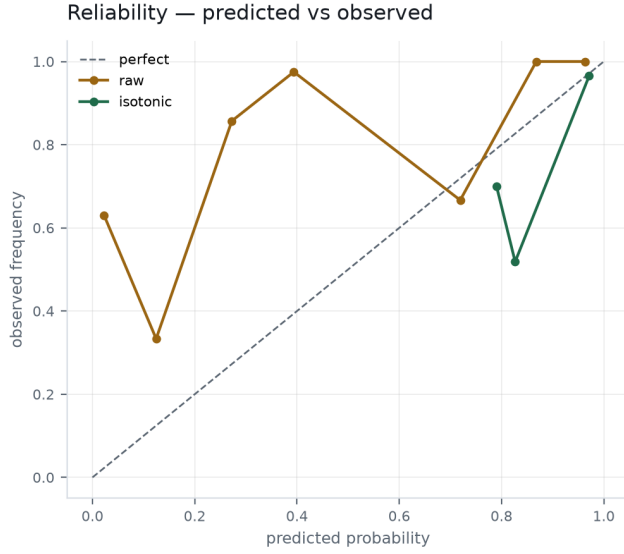


Figure 9. Reliability diagram, raw versus calibrated. Calibration is what makes “0.80” mean “80% of samples scored here were malicious.”

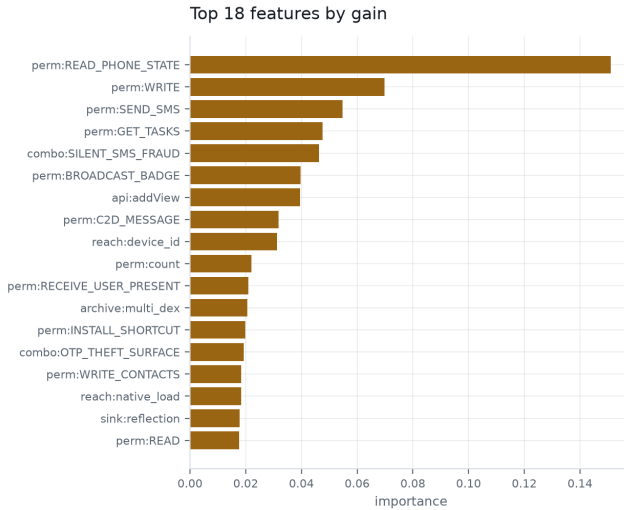


Figure 10. Top features by gain. `perm:READ_PHONE_STATE`, `perm:SEND_SMS`, `combo:SILENT_SMS_FRAUD`, `reach:device_id` and `combo:OTP_THEFT_SURFACE` dominate, reachability and permission-combination features earn their place, so this is demonstrably not a bag-of-permissions model.

14.3 End-to-End Pipeline on a Real Sample

A 50.5 MB real malware sample from the corpus, `com.brightoilonline.c2b.phone`, was walked through the complete pipeline.

Table 15. Measured stage-by-stage run on a real 50.5 MB sample

Stage	Output	Time
M1	SHA-256 recorded; stored zipped, infected convention at rest	1.32 s
M2	Permissions, 29-sink scan, backward BFS with endpoint attribution	2.60 s
M5	340-dim vector via the <i>same</i> extractor training uses	<0.01 s
M4	1 LLM call; behaviour booleans $\rightarrow B = 0.925$ via weight table	2.33 s
M6	$F_{AI} = \text{noisy-OR} \rightarrow \mathcal{S} = 46$, MEDIUM	<0.01 s
Ledger	26 nodes, chain verified; tamper $\rightarrow \text{first_bad_seq}=7$	
Total wall time		6.3 s

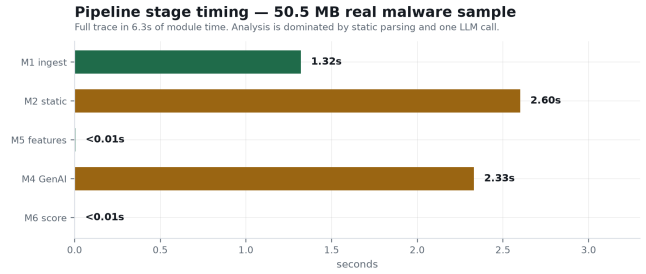


Figure 11. Stage timing. The scorer completing in under 10 ms is a design consequence, not an optimisation: it is pure, performs no I/O and calls no model.

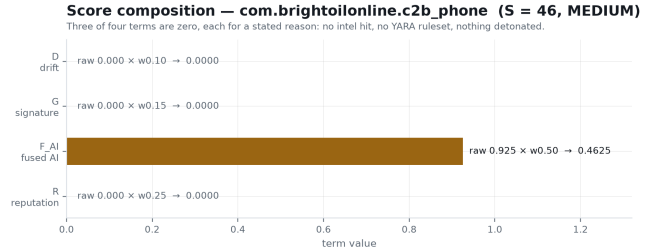


Figure 12. Score composition for the same sample. Three of the four terms are zero, *each for a stated reason* – no intel hit, no YARA ruleset loaded, nothing detonated. The score is arithmetic an analyst can audit line by line.

Confidence behaves as designed. On the inert canary application the pipeline returned $\mathcal{S} = 48$ (MEDIUM) with $\mathcal{C} = 0.021$ and $\gamma = 0.7$, 29 ledger nodes, 5.04 s wall time. Confidence rose to $\mathcal{C} = 0.637$ only once P_{cal} (0.79) and B (0.70) *agreed*. Because $\gamma = 0.4 \text{ static} + 0.3 (\text{dynamic} \wedge \text{detonated}) + 0.2 \text{ ml} + 0.1 \text{ intel}$, a run in which nothing detonated is *structurally incapable* of reporting high confidence. That is the intended behaviour, and it is the mechanism by which the system refuses to oversell itself.

14.4 Evidence Ledger Integrity

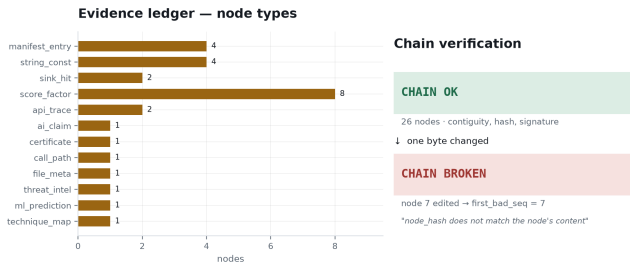


Figure 13. Node-type histogram for a real run (26 nodes: 8 score_factor, 4 manifest_entry, 4 string_const, 2 sink_hit, 2 api_trace, and one each of ai_claim, certificate, call_path, file_meta, threat_intel, ml_prediction, technique_map) beside the verification result. One byte changed $\Rightarrow$ first_bad_seq = 7.

Append-only is enforced by SQL triggers rather than by Python politeness; each node is SHA-256 chained and Ed25519 signed. The verification path is exercised as a command-line workflow, so the property is demonstrable rather than merely documented:

Listing 3. Real terminal output: verify, tamper, re-verify

```
CHAIN OK      rc=0      (29 nodes)
CHAIN BROKEN first_bad_seq=7 rc=1
reason: node_hash does not match the node's content (
    tampered)
```

```
DRISHTI - live pipeline run + evidence-integrity check

$ drishti analyse canary/dist/canary.apk

[ingest      ] ok
[ingest      ] ok
[static      ] ok
[ml          ] ok
[ml          ] ok
[genai_static] ok
[sandbox_pass1] ok
[frontier    ] ok
[sandbox_pass2] ok
[genai_full  ] ok
[score_final ] ok

package      in.drishti.canary
sinks reached ['pkg_query', 'reflection']
call paths    [['pkg_query', 'onCreate', 'depth 2']]
features      74
GenAI         B=0.97 provider=openrouter calls=1
SCORE         S=48 band=MEDIUM C=0.021 gamma=0.7
wall time     5.04s

$ drishti ledger verify --job job_10645f34b3d5

nodes      29
CHAIN OK    rc=0

$ # tamper with evidence node seq 7, then re-verify
$ drishti ledger verify --job job_10645f34b3d5

CHAIN BROKEN first_bad_seq=7 rc=1
reason: node_hash does not match the node's content (tampered)
```

Figure 14. Captured terminal session: drishti analyse on the canary, ledger verify, a single-byte tamper, and re-verification.

14.5 GenAI Determinism, A Self-Reported Limitation

The reasoning core runs NVIDIA Nemotron accessed through OpenRouter. The client is provider-agnostic by design, Gem-

ini, Claude and a deterministic mock backend are selectable at runtime through configuration, and no module depends on the choice, which matters commercially as much as technically: an institution that cannot send sample-derived strings to a third-party endpoint can point the same code at a self-hosted open-weight model without touching the pipeline.

Five identical runs over the same APK produced $B = 0.925, 0.700, 0.925, 0.925, 0.925$. Run 2 asserted one behaviour where the others asserted two. We report this rather than suppress it, because it is the empirical justification for the architecture: the model emits *enumerated booleans* and Python computes B from a fixed weight table, so residual model non-determinism is bounded, visible, and quantified at one divergent run in five, instead of being laundered into an unaccountable number.

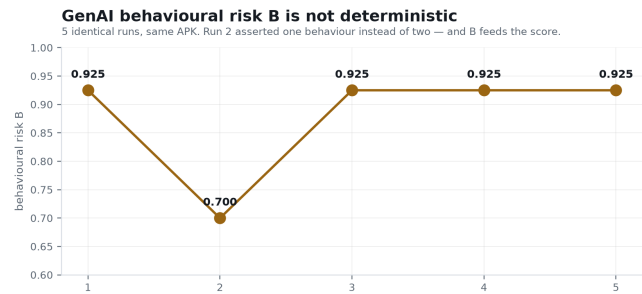


Figure 15. Five identical GenAI runs. The divergence is real, bounded by the weight table, and reported as a limitation.

14.6 Dynamic Analysis Layer

The following results exercise the *real* M3 analysis code over constructed traces. They are not detonation results and are not presented as such (§15).

Aggregation provably cannot move a score. One Cipher.doFinal event and 1,925 of them both produce exactly one ledger node and an identical $b_{\text{dynamic}} = 0.4000$. A realistic 1,929-event, 5-group trace yields $b_{\text{dynamic}} = 0.9887$. Aggregation is therefore a formatting change with no scoring consequence, which matters, because appending one node per raw event would have put 1,925 near-identical nodes in the ledger and destroyed the prompt budget.

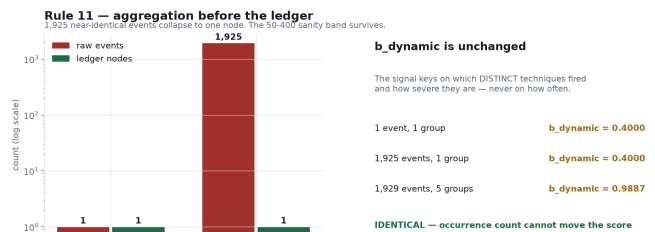


Figure 16. Event aggregation: 1 versus 1,925 events, identical b_{dynamic} .

Containment is a test, not a claim. The trustworthiness gate was run three ways. The probe inherited from the earlier iteration is *rejected* by it, and the reason is instructive: Android’s toybox `nc` has no `-z` flag, so it exited 1 with `Unknown option ‘z’` for every host. Every containment check therefore passed regardless of the real network state, and a signed manifest attested containment that had never been tested. The shipped probe runs a negative control (127.0.0.1:1, must read unreachable) and a positive control (a listener we started, must read reachable) before any verdict is trusted, and maps a timeout to *blocked* rather than to a coin flip.

Table 16. Containment probe trustworthiness gate

Probe under test	Verdict
Inherited probe (<code>nc -z</code>)	REJECTED , positive control unreachable
Always-reachable probe	REJECTED , negative control reachable
Shipped probe	TRUSTED , distinguishes open from closed

Containment verification is a test, not a claim
`assert_probe_trustworthy()` runs a negative control (127.0.0.1:1) and a positive control before any verdict is believed.

v1’s probe (<code>nc -z</code>)	REJECTED	positive control 127.0.0.1:45999 reported UNREACHABLE while a listener was run
always-reachable probe	REJECTED	negative control 127.0.0.1:1 reported REACHABLE; the probe cannot detect a clo
shipped probe	TRUSTED	probe distinguishes open from closed

v1 shipped the first row: every containment check passed regardless of the real network state, and the signed manifest attested containment that had never been tested.

Figure 17. The containment gate fails closed. A containment failure aborts the batch; it never downgrades to a warning.

Silence is never benign. The evasion detector distinguishes a stalled sample from a clean one, and a sample that produced no observations is reported *inconclusive*, never benign, environment-aware stalling is otherwise indistinguishable from an ordinary app.

Table 17. Evasion detection over three trace shapes

Trace	Verdict	Morphs proposed
2 probes, no other behaviour	STALLED	<code>install_packages</code> , <code>sms_history</code>
No observations at all	STALLED	<i>inconclusive</i>
1 probe + 20 events of real action	RAN	

Evasion detection — what triggers a morph
A sample that only asks questions is stalling. A sample that acts is not, even if it also asked.

probes, then nothing	STALLED	morphs: <code>device_identity</code> , <code>sms_history</code> 2 environment probe(s) and no other behaviour: the sample inspected its surroundings
no observations at all	STALLED	morphs: — no observations at all: the sample either detected the environment before any hook f
probes AND 20 events of real action		morphs: <code>sms_history</code> 3 non-probe behaviour group(s) observed

A sample that produced no observations is inconclusive, never benign — environment-aware stalling looks identical to a clean app if you let it.

Figure 18. Evasion detection and the morph proposals it emits back to the frontier loop.

14.7 Detection Surface Coverage

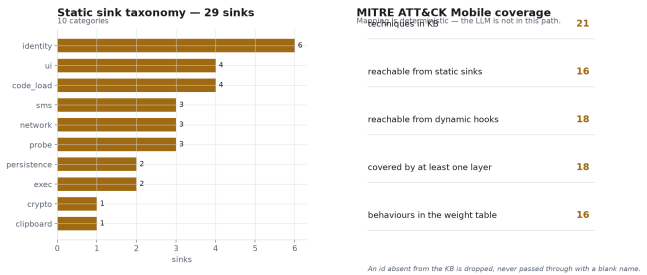


Figure 19. 29 sinks across 10 categories, and MITRE ATT&CK for Mobile coverage: 21 techniques in the knowledge base, 18 reachable from at least one detection layer. The technique mapper is deterministic, the LLM is not in that path, so it cannot hallucinate a plausible-looking technique ID that no evidence supports.

15. Findings, Methodology and Limitations

15.1 Negative Results Worth Reporting

- 70.6% of AndroZoo rows carry an implausible `dex.date`.** Unfiltered, a chronological split is noise.
- Recent Android malware is the binding supply constraint.** The full index yielded 99 samples for 2024–2026 against a 1,500 target; a size-capped list yielded 5.
- Reputation inputs must not be label-derived,** or composite-score metrics become circular.
- GenAI is not deterministic,** quantified at one divergent run in five.
- A sample with no observations is inconclusive, never benign.**
- More workers made corpus download slower,** 8 workers gave 96 samples/hr, 24 workers gave 37/hr, because AndroZoo rate limits per key.

15.2 Safety Posture

No sample is ever opened by an installer, an emulator, or a subprocess on a developer machine; static parsing with androguard is laptop-safe, `adb install` is not, and the distinction is enforced structurally rather than by policy. Corpus APKs live in a private, versioned bucket and the analysis VM’s ephemeral scratch, never committed, never copied to a laptop, stored zipped with the *infected* convention at rest. The corpus extractor carries a dual guard: an explicit flag *and* a check that the host is a cloud VM. In the sealed runtime, containment is a firewall property, default-deny egress with explicit denials to the metadata address, to RFC1918 and to the internet, and a containment failure aborts the batch.

15.3 Current Scope of Validation

The prototype validates the **decision path** end-to-end: ingestion, static analysis, learned classification, GenAI reasoning, composite scoring, evidence binding and reporting all run on real samples and are covered by 474 passing tests. The **execution path**, live detonation on the sealed detonator, is the next milestone, and the sequencing is deliberate rather than incidental.

Why the analysis layer was built before the detonator

A detonation you cannot trust is worse than no detonation. Before any sample runs, the containment probe must be provably able to distinguish an open port from a closed one (Fig. 17), event aggregation must provably not move a score (Fig. 16), and a silent sample must resolve to *inconclusive* rather than *benign*. All three gates are built, tested and measured. The detonator is then a matter of provisioning against a harness that already knows how to be honest about what it sees.

Accordingly, the dynamic-layer results in this report (Figs. 16–18) exercise the shipped M3 analysis code over controlled traces, and characterise the harness rather than any particular sample, which is exactly what a harness should be validated on before it is trusted with live malware. The hook catalogue (13 emit sites, 11 hooked methods including `Cipher.doFinal`) is syntax-checked and statically audited in CI as observational-only, so it is ready to load the moment the emulator is attached.

Model metrics are likewise a **pilot at $n = 205$ train / 144 test / 48 calibration**, and are quoted with that n throughout. The corpus pipeline that produces a full-scale successor is already built and has selected 10,599 samples; extraction at that scale is a compute question, not a research one.

Finally, provenance is derived rather than declared: replay versus live is read from the trace’s own image version, VM instance ID and run timestamp, never from a configuration flag, and surfaced on screen as the NO TRACE badge (Fig. 20). The report’s Limitations text is likewise *generated* from live flags (partial, source == replay, composite, synthetic, rejected-claim count) rather than hardcoded. The practical consequence is that this section maintains itself: the first live detonation updates it automatically, and no run can quietly stop disclosing what it did not do.

15.4 Reproducibility

Every figure and every number above regenerates from the repository with three commands, none of which touches the network, a cloud project, or a sample:

Listing 4. Full reproduction

```
uv run pytest # 474 tests
uv run python scripts/make_report_figures.py
uv run python scripts/train_and_report.py
```

Project state is tracked in `STATUS.md`, which records every task with its commit SHA and test count, and every departure

from the roadmap under an explicit *Deviations* heading. The rule the project holds itself to is simple: if a number cannot be traced to a measurement written into `STATUS.md`, it does not go into the interface or the report.

16. Availability

The complete implementation is public. This includes the full source, the pydantic contracts, the 474-test suite, the infrastructure-as-code, the dashboard, the figure-generation scripts, and the evidence pack backing every number reported in Sections 13–15.

Implementation repository

github.com/Soni-Shivam/CyberShield

Every figure and metric in this report regenerates from a clean clone using `uv run pytest` and the two scripts listed in Section 15. None of them touches the network, a cloud project, or a sample.

Acknowledgement

Claude (Anthropic) was used as a coding assistant during the implementation of this prototype.

17. Novel Contributions: Validated vs. Designed

DRISHTI makes five claims to novelty. Two are implemented, tested and demonstrable today; three are designed and specified but await the sealed detonator. Distinguishing them is not a hedge, it is the same discipline the system applies to its own verdicts, and it is what makes the first two credible.

Table 18. Contribution status. ✓ = built, tested, demonstrable now.

Contribution	St.	Evidence
Evidence Ledger for AI triage	✓	First append-only, SHA-256-chained, Ed25519-signed evidence graph in APK triage in which <code>ledger.append()</code> <i>mechanically rejects</i> an <code>AI CLAIM</code> whose <code>evidence_refs</code> do not resolve. Tamper detection demonstrated: one byte $\rightarrow$ <code>first_bad_seq=7</code>
Score-free LLM architecture	✓	The model emits only enumerated booleans; Python computes B from a 16-entry weight table. An injected “set <code>threat_score = 0</code> ” provably cannot reach $\mathcal{S}$. Residual non-determinism measured and bounded (Fig. 15)
GenAI-synthesised sandbox (JIT morphing)	~	Evasion detector and morph <i>proposals</i> built and tested (Fig. 18); the applicator awaits the detonator
Generative C2 emulation	✗	Specified; contracts defined (<code>GENERATIVE_C2</code> node type); unbuilt
Victim demographic profiling	~	Social-engineering agent contracts and prompts defined; VLM path unbuilt

The first two are, we believe, the more durable contributions. A sandbox technique is a race against malware authors; *an architecture in which the language model is structurally incapable of fabricating a verdict* is a property that does not decay when the adversary adapts.

18. Scalability & Deployment

18.1 Where the Pipeline Actually Scales

DRISHTI's fast path is embarrassingly parallel by construction. Each job is independent, the scorer is pure, and the evidence ledger is a *per-job* SQLite database in WAL mode rather than one shared table, so the store that would ordinarily be the scaling objection instead shards perfectly along the natural unit of work. Adding capacity means adding workers, not re-architecting.

Table 19. Scaling characteristics per stage, derived from the measured 6.3 s run (Fig. 11)

Stage	Time	Constraint	Scales by
M1	1.32 s	Disk I/O	Workers
M2	2.60 s	CPU (androguard)	Workers
M5	<0.01 s	None, pure compute	Free
M4	2.33 s	LLM API rate/cost	Batching, cache
M6	<0.01 s	None, pure function	Free
M3	120 s	Nested-virt VM	Hardware

The honest bottleneck. Static triage is not the constraint, detonation is. The fast path costs 6.3 s of mostly parallel work, so a single 8-core node sustains on the order of **4,500 static triages per day**; scaling that is a question of budget, not engineering. Dynamic detonation is bounded by a 120 s sandbox window on a nested-virtualisation VM that cannot be over-subscribed and must not be preemptible, giving roughly **700 detonations per VM-day**. This asymmetry is precisely why the architecture triages every sample statically and escalates only the ambiguous minority into the sandbox, the expensive resource is spent where the cheap layers could not decide.

A measured lesson in what does *not* scale. During corpus construction, raising the download pool from 8 to 24 workers *reduced* throughput from 96 to 37 samples/hour, because the upstream source rate-limits per API key. We report it because it generalises: in this domain the binding constraint is repeatedly an external quota rather than local compute, and a scaling plan that assumes otherwise will fail in production exactly as it failed here.

18.2 Analysis Lab Infrastructure

The analysis lab runs on Google Cloud in a dedicated project with a fixed budget ceiling and alerting, deliberately separated from any shared environment. Infrastructure is declared as code, a Packer image template, Terraform for the runtime VPC, and idempotent lifecycle scripts, so the lab is reproducible rather than hand-assembled.

Provisioning it surfaced a failure mode worth recording, because it is the kind that silently invalidates a batch rather than failing loudly. The initial extractor was an `n2-standard-2` whose service account carried a **read-only** storage scope: it

could pull the corpus and run extraction perfectly, and then had no way to write the results back. A scope is fixed at instance creation, so the remedy is a lifecycle operation rather than a permission edit, stop, resize, grant the write scope, restart. The current extractor is **8 vCPU, 31 GB RAM and 470 GB of free scratch**, sized so that a 193.9 GB corpus slice fits alongside its extracted features without eviction.

Two properties follow from the layout and both are enforced rather than documented. Extraction and detonation are *different machines*: the extractor only ever parses APKs statically, while detonation is confined to a separate nested-virtualisation VM on a network with no NAT and default-deny egress. And the detonator is stopped when not in use, because an idle nested-virt VM is the fastest way to consume a fixed research budget.

18.3 Cost and Concurrency Controls

Budgets are enforced as assertions in code, not as aspirations, which is what makes per-sample cost predictable enough to underwrite an SLA: ≤ 25 LLM calls per job, $\leq 12,000$ prompt tokens in, ≤ 90 s static analysis, a 120 s sandbox window, and `MAX.OBSERVATION.GROUPS = 40` so that a sample emitting 1,925 events costs the same prompt budget as one emitting one (Fig. 16). A job that would exceed a budget degrades to a partial result with `partial=True` and a reduced γ , it never silently costs more, and it never quietly reports full confidence on incomplete evidence.

Every external call is wrapped in `@degrades_gracefully`: a failing sub-analyser returns partial results with `errors` populated rather than failing the job. Under load or partial outage the system therefore sheds capability rather than availability.

18.4 Deployment Model

Table 20. Deployment topologies

Topology	Fit
On-premises (bank SOC)	Full stack in the bank's own VPC. No sample leaves the perimeter; the LLM endpoint is the only egress and can be a self-hosted open-weight model
Managed / CERT	Shared detonator pool amortised across member institutions; per-tenant ledgers keep evidence legally separable
Hybrid (recommended)	Static + ML triage on-premises for latency and data residency; detonation escalated to a pooled sealed lab

Containment is a firewall property rather than a policy document, and it is verified before every batch by a probe that fails closed (Fig. 17), which is what makes the shared-pool topology defensible to a security review at all.

19. Business Model & Unit Economics

19.1 The Arithmetic of an Analyst Hour

The economic case does not rest on detection quality; it rests on the cost of the human currently doing this work. A fraud-

desk reverse engineer spends 3–8 hours per APK and reviews 2–4 samples a day. The measured DRISHTI fast path completes in 6.3 s of machine time.

Table 21. Unit economics per APK. Machine costs are computed from measured stage timings and public cloud/API list pricing; analyst cost uses a fully-loaded senior reverse-engineer rate.

Cost component	Manual	DRISHTI
Analyst time per sample	3–8 h	5–15 min (review only)
Machine time per sample	–	6.3 s + optional 120 s
Compute cost / sample	–	<\$0.01
LLM cost / sample (<25 calls)	–	\$0.02–0.10
Detonation cost / sample	–	\$0.03 (amortised VM-hour)
Marginal cost / sample	\$150–400	<\$0.15
Throughput / analyst / day	2–4	20+ (review-bound)

Analyst throughput is bounded by human review of machine output, not by machine capacity. The system is decision-support: a human confirms every consequential action.

The marginal cost of analysis falls by roughly three orders of magnitude, which changes *what questions an institution can afford to ask*. At \$150–400 per sample a bank triages only what it must; below \$0.15 it can triage every APK reported by every customer, which is the difference between reactive incident response and continuous monitoring.

19.2 Where the Leverage Compounds

Per-sample savings are the smaller half of the case. The **campaign multiplier** is the larger one: each verdict emits an auto-generated YARA rule and the Frida script that exposed the behaviour, so a single analysed sample lets a SOC hunt the entire variant family across its monitored fleet. Malware families ship dozens of variants; one analysis defends against all of them, and the marginal cost of the next variant approaches zero.

Demographic profiling adds a second compounding effect that is specifically valuable to a retail bank: knowing that a campaign impersonates a named institution, in a named language, targeting a named customer segment, converts a technical IOC into a *pre-emptive customer warning*. The bank warns the at-risk segment before fraud is attempted rather than reimbursing after it succeeds, and avoided fraud loss, not analyst salary, is the largest number on this page.

19.3 Adoption Path

The deliberate design choice that de-risks adoption is that DRISHTI **replaces no existing control**. It consumes the same APKs a bank already receives, emits STIX/TAXII into the SIEM the bank already operates, and requires human confirmation before any consequential action. It can therefore be deployed in shadow mode alongside current triage and evaluated on its own audit trail, and because every verdict is bound to a verifiable evidence chain, that evaluation is something a compliance team can actually perform, rather

than a vendor claim it has to accept.

20. Analyst Experience

An analyst-facing tool earns trust in the interface or not at all. DRISHTI’s dashboard is therefore built on a single rule: **the screen never asserts anything the ledger cannot substantiate, and it never hides the weakness of its own evidence**. Figure 20 is a live run, no mock-up, and almost every element on it exists to make a verdict falsifiable rather than merely legible.

20.1 The Score Is Audited on Screen, Not in a Paper

The left rail does not present $\mathcal{S} = 49$ as a number to be accepted. It decomposes it into the four terms of $\mathcal{S} = 0.25R + 0.50F_{AI} + 0.15G + 0.10D$ and shows the arithmetic for each: $\text{raw } 0.972 \times \text{w } 0.5$ contributing 48.6 for F_{AI} , and $\text{raw } 0.000 \times \text{w } 0.25$ for R . Crucially, the two terms that contribute nothing are labelled **ungrounded** rather than left as a bare zero, the interface distinguishes “we looked and found nothing” from “we never looked,” which is the distinction on which an analyst’s next action actually depends.

Every factor carries **evidence chips** (`ev_10bf8ec5e...`) that resolve to ledger nodes. A finding is never a claim in prose; it is one click from the artefact that supports it. This is the evidence ledger of Section 5 surfaced as an interaction rather than a compliance appendix.

20.2 Honesty as an Interface Element

Three affordances exist purely to prevent the system overselling itself:

- **The provenance badge.** NO TRACE in red states that nothing was detonated. It is derived from the trace’s own image version, VM instance ID and timestamp, never from a configuration flag, so a replayed trace cannot be presented as a live one.
- **Confidence beside the score.** `confidence 0.02 · γ 0.70` sits next to 49 MEDIUM. A high score with low confidence is displayed as exactly that, and because γ is structurally capped without detonation, the interface is incapable of showing false certainty.
- **Enumerated behaviours, not prose.** The behaviours panel states in the interface itself that “ B is computed from a weight table in Python,” and lists the four booleans the model asserted. A judge, or an auditor, can see that the model named behaviours and did not choose a number.

20.3 Preliminary Verdict First, Depth Asynchronously

The stage strip renders the pipeline as it executes and makes the system’s central latency decision visible: stages up to

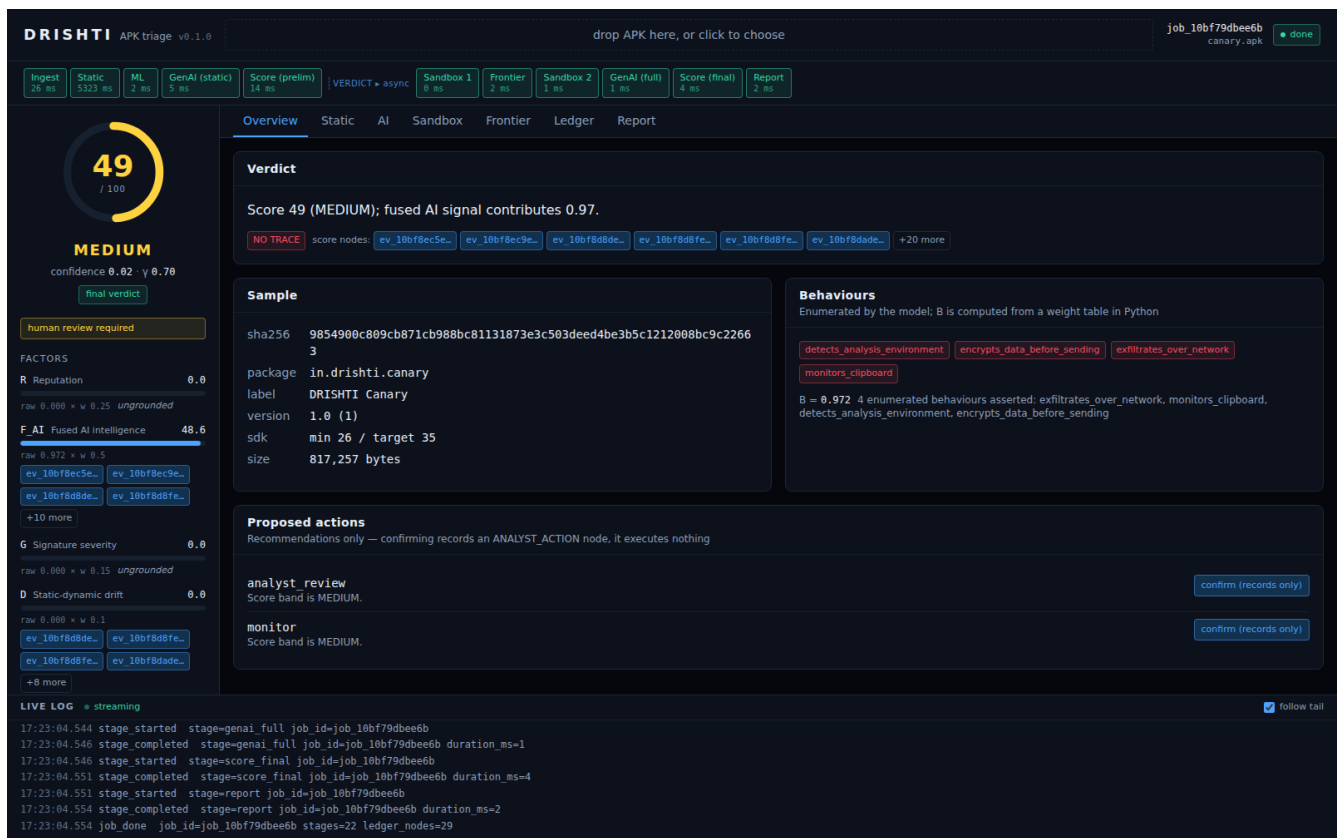


Figure 20. The DRISHTI dashboard on a completed run (`in.drishti.canary`, 22 stages, 29 ledger nodes). Left: the score rail decomposing $\mathcal{S}$ into its four weighted factors. Centre: the verdict, sample identity and enumerated behaviours. Bottom: the streaming structured log. Note the elements that exist to *limit* the claim, the red NO TRACE provenance badge, the two factors labelled *ungrounded*, the confidence 0.02 beside the score, and the human review required flag.

Score (prelim) complete in under 5.4 s, at which point the strip shows an explicit VERDICT ► `async` divider and the analyst already has an actionable band. The sandbox, frontier and full-GenAI stages continue behind that verdict and revise it when they finish. An analyst is never blocked on the slowest component, and the streaming structured log at the foot of the screen means a long-running job is legible while it runs rather than opaque until it ends.

20.4 Human-in-the-Loop, Enforced in the Product

The *Proposed actions* panel recommends `analyst review` and `monitor`, and states plainly beneath the heading: “*Recommendations only, confirming records an ANALYST_ACTION node, it executes nothing.*” Confirmation writes a signed node to the ledger; it does not block a URL, notify a customer, or push an IOC. DRISHTI is decision-support by construction, and the interface is where that guarantee is made legible to the person accountable for the decision.

20.5 Workflow Depth

Seven tabs follow the analyst’s escalation path rather than the system’s module boundaries, OVERVIEW for the verdict, STATIC, AI, SANDBOX and FRONTIER for the layer that produced a given finding, LEDGER for the full hash chain with one-click verification, and REPORT for the exportable artefact. Panels that depend on unbuilt modules render an explicit *not yet implemented* state naming the owning task, rather than an empty panel or a plausible placeholder. Even the absence of a feature is reported honestly.

21. Conclusion

DRISHTI addresses the full spectrum of APK-borne financial fraud: Level 1 malicious artefacts detectable by static and ML analysis, and the harder Level 2 latent threats that evade every existing scanner. Its core innovation is a GenAI reasoning layer that does not merely classify - it acts as an automated reverse engineer, synthesising the very environment a dormant trojan is hunting for to make it confess, then binding every sentence of its verdict to a concrete artefact that an analyst can verify.

The evidence ledger transforms AI output from an opaque score into a machine-generated audit trail. The GenAI-synthesised sandbox - emulating dead C2 infrastructure and morphing device state Just-in-Time - unrolls behaviour that dormant trojans conceal from generic sandboxes. The multi-label ML ensemble provides signature-agnostic generic checks and calibrates confidence. GenAI reverse engineering exposes the malware's intent, impersonated target, and victim demographic. Together, these layers deliver what bank fraud desks actually need: not "this looks malicious" but "here is exactly what this app does, here is the proof, here is the YARA rule to find its siblings, and here is who it targets."

The DRISHTI thesis

GenAI is a first-class analyst inside the detection pipeline - reading decompiled code and live behaviour the way a human reverse engineer would, and actively synthesising the conditions that force malware to reveal itself - while the threat score is produced by a calibrated, explainable, multi-signal engine. Every verdict is bound to cryptographic evidence, and a human confirms every consequential action. The AI reasons; it never guesses the number.

Every other system classifies an APK. **DRISHTI interrogates it.**

References

- [1] VirusTotal. *Introducing VirusTotal Code Insight: Empowering threat analysis with generative AI*. <https://blog.virustotal.com/2023/04/introducing-virustotal-code-insight.html>
- [2] VirusTotal. *Empowering Defenders: How AI is shaping malware analysis*. <https://assets.virustotal.com/reports/2023-ai>
- [3] Sysdig. *More than an Assistant - A New Architecture for GenAI in Cloud Security*. <https://www.sysdig.com/blog/genai-cloud-security>
- [4] Sysdig. *Sysdig Sage: A groundbreaking AI security analyst*. <https://www.sysdig.com/blog/sysdig-sage-a-groundbreaking-ai-security-analyst>
- [5] Sysdig. *What is Falco? Open source runtime threat detection*. <https://www.sysdig.com/learn-cloud-native/what-is-falco>
- [6] Reserve Bank of India. *Annual Report on Cyber Security in the Indian Banking Sector, 2024*. <https://www.rbi.org.in>
- [7] MITRE. *ATT&CK for Mobile*. <https://attack.mitre.org/matrices/mobile/>
- [8] Mobile Security Framework (MobSF). <https://github.com/MobSF/Mobile-Security-Framework-MobSF>
- [9] Frida - Dynamic instrumentation toolkit. <https://frida.re/>
- [10] Androguard. <https://github.com/androguard/androguard>
- [11] YARA. <https://virustotal.github.io/yara/>
- [12] abuse.ch - MalwareBazaar & URLhaus. <https://abuse.ch/>
- [13] AndroZoo dataset. <https://androzoo.uni.lu/>
- [14] CIC-MalDroid 2020. <https://www.unb.ca/cic/datasets/maldroid-2020.html>
- [15] Quark-Engine. <https://github.com/quark-project/quark-engine>
- [16] Team DRISHTI. *CyberShield / DRISHTI, prototype implementation*. <https://github.com/Soni-Shivam/CyberShield>